



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No. : 12

Team No.: 9

Project Title: Linux Memory-Mapped File Management System

Team Members

Roll Number	Student Name	Signature
2520080013	D. Sri Varshini	
2520080001	Shreyas Panjala	
2520080022	Ch Karthikeya	

1. Abstract

The **Linux Memory-Mapped File Management System** is a Linux-based system programming project designed to demonstrate an efficient method of accessing and managing files through memory mapping. Traditional file handling requires repeated use of read and write operations, which can involve additional system calls. This project addresses this by using the `mmap()` system call to map a file directly into the process's virtual address space, allowing the file contents to be accessed and modified as memory.

The project focuses on key **Operating Systems and System Programming concepts**, including virtual memory, memory mapping, process address space, file management, system calls, memory protection, and synchronization. The proposed system provides a simple interface through which users can open files, map them into memory, read and modify their contents, synchronize changes using `msync()`, and safely remove the mapping using `munmap()`.

The system will be implemented in **C on a Linux environment**, using standard Linux system calls such as `open()`, `fstat()`, `mmap()`, `msync()`, `munmap()`, and `close()`. The expected outcome is a functional application that demonstrates how Linux connects file storage with virtual memory and provides efficient file access through memory mapping. The project will also provide practical understanding of how operating systems manage memory and files at the system-call level.

2. Problem Statement

(Clearly describe the problem that the project intends to address and its relevance to Operating Systems and Systems Programming.)

Traditional file management in Linux requires repeated system calls such as `read()` and `write()` to access and modify file contents. Frequent file operations can increase system-call overhead and require separate memory buffers for processing data. The project aims to address this problem by developing a **Linux Memory-Mapped File Management System** using the `mmap()` system call, which maps file contents directly into a process's virtual address space. This allows the program to access and modify file data through memory while reducing the need for repeated read/write operations. The project is relevant to **Operating Systems and Systems Programming** because it demonstrates important concepts such as virtual memory, address-space management, file management, memory protection, system calls, and synchronization. The system provides a practical way to understand how Linux manages the relationship between files and process memory.

3. Objectives

(List the specific objectives of the project.)

1. To understand and implement **memory-mapped file management** using the Linux `mmap()` system call.
2. To provide efficient access to file contents by mapping files directly into the **process's virtual address space**.
3. To demonstrate important OS concepts such as **virtual memory, system calls, memory protection, and file management**.
4. To implement safe file operations including **mapping, reading, modifying, synchronizing, and unmapping** files using Linux system calls.

4. Proposed Methodology

(Briefly explain how the project will be designed and implemented in Linux. Describe the major functionality, approach, and flow of the proposed solution.)

The system will be developed in C on a Linux environment using memory-mapped file operations. The program will first open a selected file using `open()` and obtain its size using `fstat()`. The file will then be mapped into the process address space using `mmap()`. Users can read and modify the file contents directly through the mapped memory. The changes will be synchronized with the file using `msync()`, after which the memory mapping will be removed using `munmap()` and the file will be closed using `close()`. The overall flow demonstrates how Linux manages files through virtual memory and system calls.

5. Operating Systems Concepts / Linux APIs Used

(List the relevant Operating Systems concepts, Linux system calls, POSIX APIs, or mechanisms that will be used.)

OS Concept / Linux API / System Call	Purpose in the Project
Virtual Memory & Address Space	Provides the virtual address space used for memory-mapped files.
<code>open()</code> & <code>close()</code>	Opens the file for processing and safely closes it upon completion.
<code>fstat()</code>	Obtains the file size (<code>st_size</code>) and related file metadata.

mmap() & Memory Protection	Maps the file into memory and controls read/write access permissions.
msync() & Synchronization	Synchronizes modified memory contents back to physical storage on disk.
munmap()	Removes the mapped memory region safely from the process address space.

6. Individual Contribution

(Clearly specify the responsibility of each team member in the project.)

Roll Number	Student Name	Individual Responsibility
2520080013	D.Sri Varshini	File Handling Module: Implement file operations using open(), fstat(), and close(), and handle file-related errors.
2520080001	Shreyas Panjala	Memory Mapping Module: Implement mmap() and munmap() to map and unmap files, and handle memory protection.
2520080022	Ch Karthikeya	Synchronization & Testing: Implement msync() for synchronizing changes, integrate the modules, test the complete system, and document the results.

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system and development environment
C / C++	Implementation of the system
GCC	Compiling the C program
Linux System Calls / POSIX APIs	File and memory management
Git / GitHub	Version control and project collaboration

8. Expected Outcome

(Describe the expected functionality, output, and outcome of the project.)

The expected outcome is a functional Linux-based application that allows users to open a file, map its contents into memory, read and modify the contents, synchronize the changes with the file, and safely unmap the memory. The project will demonstrate the practical use of mmap(), msync(), munmap(), and other Linux system calls while providing an understanding of virtual memory, file management, memory protection, and process address space.

9. GitHub Repository

GitHub Repository Link: <https://github.com/ShreyasPanjala/KLH-AI-and-DS-2026-27-Team-9-Memory-Mapped-File-Management-System>

Repository Created: _____

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: M.Raghupathi

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____